



Beehive: A Scalable Disaggregated Memory Runtime Exploiting Asynchrony of Multithreaded Programs

Quanxi Li, Hong Huang, Ying Liu, and Yanwen Xia, *Institute of Computing Technology, Chinese Academy of Sciences; University of Chinese Academy of Sciences*; Jie Zhang, *Peking University*; Mosong Zhou, *Huawei Cloud*; Xiaobing Feng and Huimin Cui, *Institute of Computing Technology, Chinese Academy of Sciences; University of Chinese Academy of Sciences*; Quan Chen, *Shanghai Jiao Tong University*; Yizhou Shan, *Huawei Cloud*; Chenxi Wang, *Institute of Computing Technology, Chinese Academy of Sciences; University of Chinese Academy of Sciences*

<https://www.usenix.org/conference/nsdi25/presentation/li-quanxi>

This paper is included in the
Proceedings of the 22nd USENIX Symposium on
Networked Systems Design and Implementation.

April 28–30, 2025 • Philadelphia, PA, USA

978-1-939133-46-5

Open access to the Proceedings of the
22nd USENIX Symposium on Networked
Systems Design and Implementation
is sponsored by



Beehive: A Scalable Disaggregated Memory Runtime Exploiting Asynchrony of Multithreaded Programs

Quanxi Li^{1,2} Hong Huang^{1,2} Ying Liu^{1,2} Yanwen Xia^{1,2} Jie Zhang³ Mosong Zhou⁴
Xiaobing Feng^{1,2} Huimin Cui^{1,2*} Quan Chen⁵ Yizhou Shan⁴ Chenxi Wang^{1,2*}

¹SKLP, ICT, CAS ²University of Chinese Academy of Sciences
³Peking University ⁴Huawei Cloud ⁵Shanghai Jiao Tong University

Abstract

The Microsecond (μ s)-scale I/O fabrics raise a tension between the programming productivity and performance, especially in disaggregated memory systems. The multithreaded synchronous programming model is popular in developing memory-disaggregated applications due to its intuitive program logic. However, our key insight is that although thread switching can effectively mitigate μ s-scale latency, it leads to poor data locality and non-trivial scheduling overhead, leaving significant opportunities to improve the performance further. This paper proposes a memory-disaggregated framework, *Beehive*, which improves the remote access throughput by exploiting the asynchrony within each thread. To improve the programming usability, *Beehive* allows the programmers to develop applications in the conventional multithreaded synchronous model and automatically transforms the code into *pararoutine* (a newly proposed computation and scheduling unit) based asynchronous code via the Rust compiler. *Beehive* outperforms the state-of-the-art memory-disaggregated frameworks, *i.e.*, Fastswap, Hermit, and AIFM, by $4.26\times$, $3.05\times$, and $1.58\times$ on average.

1 Introduction

Memory disaggregation allows applications to utilize the available memory of other servers and acquire orders of magnitude higher memory resources to avoid performance degradation due to insufficient memory, such as swapping data to disk. However, accessing remote memory exhibits microsecond (μ s)-scale access latency, which incurs non-trivial performance penalties over accessing local memory [15, 60, 88, 108, 109]. Hence, a new challenge arises in the memory disaggregated data center—how to mitigate the μ s-scale latency of remote access.

State of the art. The conventional wisdom of mitigating data accessing overhead is to avoid CPU idle time through the multithreaded synchronous programming model [15, 56, 92, 105]. Taking the memory disaggregated platform, AIFM [92], as an example: when accessing remote data, the thread will be suspended after issuing a request and then yield the core to

another ready thread immediately, as shown in Figure 1(a). To further improve CPU efficiency, a series of kernel-bypass techniques have been proposed to alleviate the thread context switching overhead, such as Shenango [82], AIFM [92], and ZygOS [87]. The user-level thread (referred to as *uthread*) context switching consumes only around 200 cycles for context saving and restoration [82], $10\times$ faster than kernel thread (*i.e.*, POSIX thread, referred to as *kthread*), which helps quickly fill the idle CPU by another ready task during the μ s-scale remote access.

However, we observed some overlooked performance penalties for applications with an increasing number of *uthreads*. Intuitively, the μ s-scale remote access latency, *e.g.*, reaching up to 6 μ s [88, 92], easily outweighs the *uthread* switching overhead. Hence, launching a large number of *uthreads* to saturate the CPU resources and achieve high remote memory access concurrency is necessary and profitable. However, a series of consequences caused by thread switching leads to serious performance loss.

Taking Multi-Grid PDE [65] as an example (§ 3). When running on the state-of-the-art remote memory data plane, AIFM [92], the application’s performance is still throttled by two main reasons—first, frequent *uthread* switching worsens data locality, leading to much higher CPU cycles spent on local memory access due to significantly enlarged LLC misses. Second, the scheduling overhead takes up to 23% of the CPU time, which becomes worse as the number of *uthread* increases or as the local memory size shrinks.

In other words, hiding remote access latency via *threading* can stealthily consume non-trivial CPU resources due to poorer data locality and heavier scheduling, which shifts the bottleneck from remote memory access to local memory and *uthread* managing. Although tuning the number of *uthreads* can alleviate the aforementioned inefficiency, the sweet spot of each application can be different (§ 5.4), which usually incurs unpredictable tuning and profiling overhead.

Major insights. The main question is, *can the application generate enough concurrent remote memory accesses to mitigate the μ s-scale latency and also provide good data locality to avoid excessive cache misses?*

On the one hand, improving multithreaded program’s data locality has been extensively studied for decades [36, 58,

* Corresponding authors: Chenxi Wang and Huimin Cui.

72, 94]. A series of locality-aware memory allocators have been proposed and integrated into the existing systems. For example, they segregate the objects of different threads via the thread local allocation buffer (TLAB) to avoid false sharing [19, 40, 59] and aggregate the adjacent allocated objects for a better cache/page locality [58]. Therefore, letting a core execute a thread without preemption can largely maintain good data locality due to substantial accessing data in the TLAB.

On the other hand, we find a mass of opportunities to parallelize the code of big data applications. The code of a thread can be split into a series of tiny code blocks (*e.g.*, coroutines [5, 8]) without data dependency, which enables asynchronous execution within a thread —when encountering remote data access, the subsequent coroutines can continue execution without blocking on waiting for the completion of previous remote access, as shown in Figure 1(b). Compared to thread switching, the routine-based asynchronous execution has much smaller working set size (WSS) and context switching overhead, which makes it a good fit for achieving high concurrency.

Beehive. Based on the aforementioned observation, we propose Beehive, an asynchronous framework designed for memory-disaggregated servers. Instead of relying on thread (uthread or kthread) switching to hide the μ s-scale latency, Beehive lets each application thread **run to completion** [16] to maintain well-organized data locality and exploit high CPU efficiency and remote access concurrency via coroutine-based **asynchronous execution model** [9, 30, 84, 86]. To eliminate the well-known programming obstacles of asynchronous programming [15], such as *stack ripping* [9] and *callback hell* [56], Beehive allows developers to write programs in the conventional synchronous model with provided APIs and automatically transform the programs to the asynchronous model via the Rust compiler (§ 4).

Beehive’s design targets two main challenges:

First, *how to transform synchronous code to asynchronous model automatically?* For code transformation, the compiler needs to split the code into a series of coroutines and then schedule the generated routines based on the program dependency analysis. However, pointer aliasing remains the major obstacle for data dependency analysis. To tackle this challenge, Beehive is implemented in Rust to benefit from its *ownership* types that provide *uniqueness* and *immutability* constraints during programming, which helps manage the pointer aliasing issues [41, 76] (§ 4.3).

Second, *how can we efficiently mitigate μ s-scale remote memory access latency via coroutines?* A large number of concurrent remote accesses is necessary. Hence, first, Beehive implements a lightweight execution and scheduling unit, *pararoutine*, based on the Rust coroutine [8] to support high concurrency. The switching (yielding the core and scheduling another task to run) of a pararoutine is $2.3\times$ faster than Shenango uthreads due to the stackless design and a cache-

locality-optimized allocator that needs only five instructions to allocate a routine frame. Second, pararoutines employ cooperative scheduling for batch processing applications to contain the scheduling overhead while providing preemption for the latency-critical applications to keep the low tail latency (§ 4.4). Note that Beehive also supports thread switching when it cannot generate enough pararoutines to fill idle CPUs.

In summary, Beehive outperforms state-of-the-art solutions, *e.g.*, AIFM [92] and Hermit [88], through better data locality, more lightweight switching, and similar concurrency.

Results. We have evaluated Beehive with eight workloads that cover a wide spectrum of memory access patterns. Experimental results demonstrate that Beehive outperforms the state-of-the-art kernel paging-based frameworks, Fastswap [11], Hermit [88], and a runtime-based framework, AIFM [92], by, on average, $4.26\times$, $3.05\times$ and $1.58\times$, correspondingly. In addition, Beehive also reduces the tail latency by, on average, $502\times$ and $13.2\times$, compared to Hermit and AIFM. Beehive is available at <https://github.com/ICTPLSys/Beehive>.

2 Background

This section presents existing disaggregated memory systems and discusses the remote memory programming models.

2.1 Runtime-based Disaggregated Memory

Disaggregated memory systems are generally categorized into three types: runtime-based [21, 24, 47, 66, 92, 106, 107], kernel paging-based [43, 88, 95, 108, 112], and hardware-based [22, 42, 45, 60, 61]. Although the paging-based solutions are transparent, they usually incur severe I/O amplification issues and lengthy remote access latency due to managing memory via the kernel paging [13, 21, 92]. On the other hand, considering that there is no commodity hardware-based solution at this moment, we implement Beehive according to the design of AIFM [92], a runtime-based framework that is built on the Shenango uthreads [82]. There are two major reasons: First, the data is managed in fine-grained object granularity, *e.g.*, *remoteable pointer* [92], to alleviate the I/O amplification and also utilize the program semantics to improve the prefetching effectiveness. Second, kernel-bypassing techniques [70, 87] help alleviate the thread switching overhead to achieve much higher remote access concurrency.

2.2 Programming Model

Compared to hardware mechanisms, *e.g.*, out-of-order execution and non-blocking caches [64], software solutions are more suitable for hiding the μ s-scale latency due to their ability to generate high remote memory access concurrency and flexible data access granularity. Remote data management can be programmed in two different ways: *synchronous model* [15, 48, 56, 92, 105] and *asynchronous model* [9, 30, 84, 86].

Synchronous Programming. When developing a synchronous program, developers think linearly about the program’s control flow—the thread will block on the I/O call site until the remote access completes, and there is no need to worry about accessing incomplete data. The remote access concurrency can be achieved via threading, which schedules another ready thread to preempt the CPU to alleviate the I/O stall (Figure 1(a)), shifting the programming burden to the thread library [105]. Most existing runtime-based solutions use the synchronous model for programming productivity and robustness [15, 24, 92].

Asynchronous programming. On the contrary, the control flow of asynchronous programs is much more flexible. As shown in Figure 1(b), after issuing the remote data access, the thread can skip the related code that may access the data being fetched and continue executing the subsequent code, hiding the remote access latency. Compared to synchronous programs, asynchronous programs can achieve similar concurrency with lower context switching overhead and better data locality due to sticking to the thread’s local data. However, the developers need to be aware of the program data dependency and manually register the skipped code as a *continuation* that needs to be processed after the pending remote access is finished, as in the FaRM [34] (a transaction-based asynchronous programming model for distributed shared memory), which is prone to error and has become one of the main challenges of adopting asynchronous programming [9, 56].

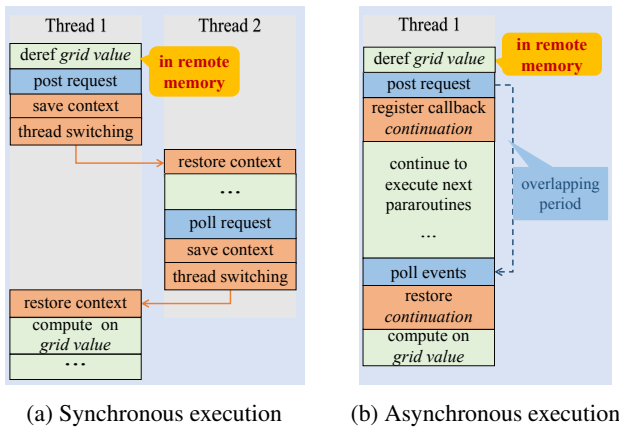


Figure 1: The (a) synchronous and (b) asynchronous execution flow of traversing grid points in Multi-Grid PDE [65].

2.3 Program Dependency Analytics

Accurate and thorough program dependency analytics is the foundation of transforming the synchronous application into the asynchronous model. Pointer aliasing remains the main obstacle to program dependency analysis [31, 38, 46, 98]. However, *ownership* types are effective in eliminating the alias issues in large-scale programs [25, 26, 69, 80] and have been widely adopted by system languages, such as Rust [69],

which gives us an opportunity to build the Beehive compiler for automatic code transformation.

Pointer aliasing means that a memory location is accessible by multiple symbolic names, such as pointers and references. The alias may escape the compiler’s static analysis. As a result, the data can be modified through some of the aliases without the compiler being aware of the aliasing. Hence, the compiler needs to conservatively treat the potential alias when converting the code into concurrent routines to avoid violating execution orders, such as read-after-write (RAW). As a result, plenty of asynchronous code transformation opportunities (*i.e.*, concurrency) are wasted in large-scale programs that have complicated pointer usage and data dependency.

Ownership. When programming with the Rust *ownership* types, the references have the *uniqueness* and *immutability* properties, which help control aliasing [69, 80]. Uniqueness ensures that an object can only have one active mutable reference at a time. Another reference can *borrow* the mutable reference from the owner for a period of time (within a scope), during which time the owner cannot modify the data. As a result, the compiler can statically capture the transition of data’s unique mutable reference and locate all the data dependencies, *i.e.*, true dependency (read-after-write, RAW), anti-dependency (write-after-read, WAR), and output dependency (write-after-write, WAW) [80], that can be used for concurrent routine generation. Note that multiple immutable references (read-only) can be created from the owner for reading, which is safe for the concurrent routine generation. Refer to § 4.3 for more details.

3 Motivation

This section demonstrates the performance bottleneck in today’s remote memory systems, which shifts from remote memory access to local memory access and thread management, and the potential benefits of exploiting the asynchrony within a uthread.

We use Multi-Grid PDE (referred to as MGP, § 5.1), a memory-intensive scientific partial differential equation solver from NAS Parallel Benchmarks (NPB) [65], as the example to demonstrate the performance breakdown. MGP performs V-cycle multi-grid method to compute a 3D scalar Poisson equation, which exhibits good spatial locality due to its iterative stencil computation on fine-grained and coarse-grained grids. We evaluate MGP on two kinds of far-memory data planes, the paging-based Hermit [88] and the runtime-based AIFM [92]. We execute the original MGP on Hermit and re-implement the MGP on AIFM by using its utthread and Shenango runtime [82]. When running on Hermit, MGP is configured to one kthread per core, while the number of application utthreads is tuned to optimal, *i.e.*, 64 utthread per core, when running on AIFM (§ 5.4). Note that we configured the same number of utthreads for its prefetching and evacuation utthread pool as AIFM does [92]. We allocate 8 cores and 34 GB memory in total, among which 13% memory is

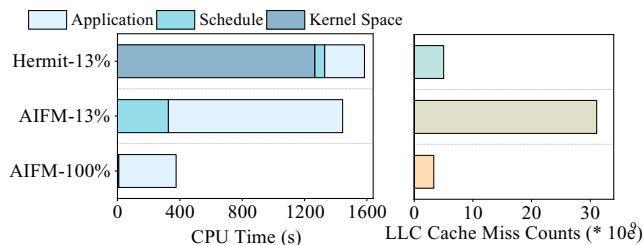


Figure 2: Performance breakdown of MGP when running on Hermit and AIFM. Although AIFM can reduce remote access overhead, it incurs non-trivial thread scheduling overhead and results in degraded application performance due to excessive LLC misses.

local and the rest is remote. The local and remote servers are connected via a pair of 100 Gbps Mellanox ConnectX-5 InfiniBand adapters.

As shown in Figure 2, Hermit and AIFM achieve similar performance, *i.e.*, AIFM slightly outperforms Hermit by 10%, while they are throttled by different reasons—(1) when running on Hermit, application throughput is throttled by the kernel paging that is used for remote access due to the page table management [12, 13], swap entry allocation [108], and other kernel space overhead [112]. Note that Hermit is the SOTA far-memory data plane and has already improved the paging efficiency for disaggregated memory; (2) AIFM is designed to address the kernel space overhead and I/O amplification [21] (which is negligible in MGP due to the good spatial locality). AIFM reduced 64% of CPU cycles spent on remote access (on-demand fetching, prefetching, and evacuation) by avoiding trapping into the kernel and efficient thread switching. However, except for the runtime bookkeeping overhead [24], we observed non-trivial thread scheduling overhead and significantly degraded application throughput due to LLC misses, which becomes the new bottleneck.

Thread switching leads to poor data locality. MGP organizes the data as two arrays, one for the fine-grained grid and another for the coarse-grained grid. The two grids are sliced into chunks and evenly assigned to different utthreads for load-balancing and high parallelism. Each utthread iterates and process the slices alternating between the fine and coarse grids. As shown in Figure 2, AIFM leads to 6.5× more LLC misses than running on Hermit due to frequent utthread switching. There are two root causes of why the multithreaded synchronous model has poor data locality.

On the one hand, the application data needs to be partitioned across application utthreads to reduce data racing, resulting in a separate data hotspot for each application utthread. At the same time, AIFM relies on frequent utthread switching to fill the CPU idle time during the μ s-scale synchronous remote accesses, which urges for a large number of utthreads and high switching frequency, resulting in a large working set size (WSS). For example, MGP requires 64 application utthreads to achieve optimal performance (§ 5.4), causing the

application utthread switching up to 4×10^8 times during the execution.

On the other hand, the runtime-based solution, *i.e.*, AIFM, is designed to manage data at fine granularity to exploit application semantics and reduce the I/O amplification [92], which further increases the demand for utthread-level concurrency. For example, AIFM maintains two separate utthread pools for object prefetching and evacuation, 17 for prefetching, and 40 for evacuation under the configuration of 8 cores [92], further increasing the number of utthread switchings.

Thread scheduling overhead. Another major performance penalty of the AIFM-based MGP comes from the thread scheduling overhead, which consumes 23% of total CPU cycles in this case. Although, the utthread context switching consumes only around 200 cycles [82], the cost of scheduling is non-trivial. During each utthread switching, the utthread needs to be paused and inserted into a waiting queue for the next step of scheduling, the cost of which can be 10× higher than the context switching. The utthread switching frequency in AIFM is high, reaching up to 2.2 million times per second. Hence, it is beneficial to continue reduce the thread context switching and scheduling overhead.

On the other hand, the large number of concurrent utthreads lead to heavy synchronization overhead, both within the utthread group and across different utthread groups. As aforementioned, MGP utilizes up to 121 utthreads under 8 cores (64 for application, 17 for prefetching and 40 for evacuation). Taking the synchronization between the application and the evacuation utthread groups as an example, the evacuator is used to concurrently compact the live objects for defragmentation. To avoid data race, a fence (*dereference scope*, § 4.4) is used to synchronize the application utthreads and the evacuation utthreads—the evacuation utthreads needs to be held until all the executing application utthreads exit their dereference scopes at least once [24, 92]. Hence, more live utthreads can potentially lead to more synchronization overhead.

Conclusions. Mitigating μ s-scale latency through frequent utthread switching is the root cause of the poor data locality and scheduling overhead, not the number of utthreads. Hence, exploiting asynchrony within a thread provides opportunities to maintain a better data locality while achieving even similar (even higher) remote access concurrency.

4 Design and Implementation

4.1 Overview

Beehive is an asynchronous framework built for applications running on the memory-disaggregated platform. Beehive is developed based on the Rust language to benefit from the *Ownership* type system and contains three layers, programming interfaces (§ 4.2), a Rust compiler (§ 4.3) and the associated runtime (§ 4.4), as demonstrated in Figure 3.

For the asynchronous execution, programmers only need to develop the application in a conventional multithreaded syn-

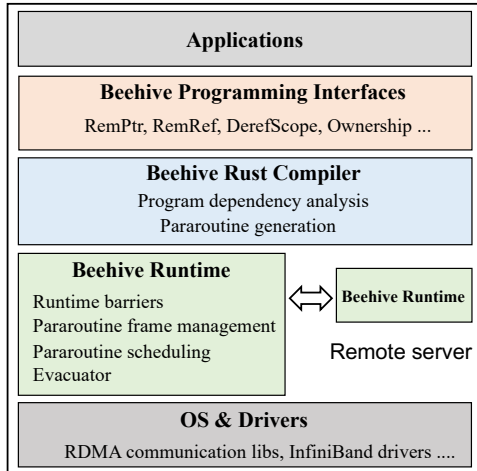


Figure 3: Design overview of the Beehive.

```

1 // remoteable pointer
2 struct RemPtr<T> { metadata: u64 /* see Figure 4 */ }
3 impl<T> RemPtr<T> {
4     fn deref(&self) -> RemRef<T>;
5     fn deref_mut(&mut self) -> RemRefMut<T>;
6     // allocate remoteable object,
7     // the position can be local, remote or offloaded
8     fn allocate(&mut self, value: T, position: Pos);
9     /* ... */
10 }
11 // remoteable references
12 struct RemRef<'a, T>; // immutable
13 impl<T> Deref for RemRef<'_, T>;
14 struct RemRefMut<'a, T>; // mutable
15 impl<T> DerefMut for RemRefMut<'_, T>;

```

Listing 1: Remote data management interfaces

chronous model via the Beehive interfaces, and the provided Rust compiler transforms the code of a thread into a series of *pararoutines* (§ 4.4.1) that can execute concurrently. Beehive compiler adds runtime barriers to support data location checks and movement between the local and remote memory. When encountering remote access, the compiler wraps the rest of the pararoutine code snippet as a *continuation* and inserted into a waiting queue after issuing the remote access request. The thread will continue to execute the subsequent code (or pararoutines) continuously to process the thread’s local data.

4.2 Programming interfaces

Listing 1 demonstrates the Beehive APIs for manipulating the remote data. There are two types of interfaces: (1) the *remoteable pointer* (`RemPtr`) that is derived from the Rust `Box<T>` type is used for allocating objects in the remoteable space (Figure 7); and (2) the *remoteable references* (`RemRef` and `RemRefMut`) that is derived from the Rust *reference* type is used for (mutable and immutable) borrowing and accessing the remote objects.

Remoteable pointer. When allocating data via the remoteable pointer (`RemPtr`), the data can be explicitly allocated in the remote memory, local memory or offloading spaces (§ 4.4.2) by setting the parameter `position`. For each `RemPtr`, there

are 64 bits of metadata recording the address and control information. As shown in Figure 4, the low 48 bits are used for address and the high 16 bits are used to record control information, including a `present` (P) bit for local and remote status, a `moving` (M) bit to indicate if the object is being moved by the evacuator, an `accessed` (A) bit and a `dirty` (D) bit for hotness tracking, eight bits to record the number of references (`ref_count` (RC)). There are also several reserved bits that can be used by the developers to customize their own hotness tracking mechanisms, such as the CLOCK replacement algorithm [101, 111] and the LRU-like algorithm [17].

1	present	:	63		1	present	:	63	// = 0
2	moving	:	62		2	moving	:	62	// = 0
3	accessed	:	61		3	offloaded	:	61	
4	dirty	:	60		4	reserved	:	60	
5	ref_count	:	59-52		5	size	:	59-48	
6	reserved	:	51-48		6	r_addr	:	47-0	
7	addr	:	47-0		7				

(a) Local object

(b) Remote object

Figure 4: The metadata layout of the remoteable pointer (`RemPtr`) for objects (a) in the local memory or (b) in the remote memory.

Remoteable pointer dereferencing. Beehive adds runtime barriers, *i.e.*, `deref` and `deref_mut`, to the `RemPtr` to dynamically check the the control bits of the remoteable pointer during execution for object management (referred to as *dereferencing*). For example, the object will be retrieved into the local memory if the `present` bit is unset, or the dereferencing will be hold if the `moving` bit is set. The barrier is optimized to minimize the execution overhead. For the hot path (object is already in the local memory), only five instructions are needed for the location check, which is comparable to AIFM [92]. For the slow path (object is in the remote memory), we use atomic instructions to modify the pointer, avoiding the cost of locking, and the overhead is trivial when compared to the μ s-scale latency for retrieving the data from the remote memory.

Remoteable reference. A `T`-typed object in the remoteable space is uniquely owned by a `RemPtr<T>`. To avoid the runtime barrier overhead of accessing data via the `RemPtr`, Beehive lets programmers borrow the data from the `RemPtr` to a zero-runtime-overhead remoteable reference, `RemRef<T>` (or `RemRefMut<T>` for mutable access), as shown in Listing 2, for subsequent accesses. During the borrowing, the dereferencing methods, *e.g.*, `deref` for immutable borrow and `deref_mut` for mutable borrow, will be invoked to inspect the control bits of the remoteable pointer and retrieve the data to local if necessary. After the one-time dereferencing (aligned with borrowing), the data will reside in the local memory and can be accessed through the zero-runtime-overhead `RemRef` (or `RemRefMut`) as that of raw pointers.

```

1 elements: [RemPtr<T>; N];
2
3 impl<T, const N: usize> RemArray<T, N> {
4     fn for_each<F>(&self, f: F) where F: Fn(&T) {
5         for e in self.elements.iter() {
6             // immutable borrow
7             let r: RemRef<T> = e.deref();
8             f(*r); // zero-runtime-overhead access
9             // end of the immutable borrow
10        }
11    }
12    fn initialize(&mut self, val: T) {
13        for e in self.elements.iter_mut() {
14            e.allocate();
15            // uniquely mutable borrow
16            let mut r: RemRefMut<T> = e.deref_mut();
17            // let r2 = e.deref(); Compiler Error
18            *r = val;
19            // end of the mutable borrow
20        }
21    }
22 }

```

Listing 2: Example code of an implementation of RemArray. The programming should follow the constraints of Rust *ownership*.

Pararoutine scope. We let developers explicitly define the pararoutine scope and rely on the compiler to finish the generation of the pararoutine, e.g., frame allocation and free variable capturing (Listing 3). Automatic parallelization (relying on the compiler to analyze and automatically split the code into concurrent pararoutines) remains a very challenging problem [28, 100, 110] beyond the scope of this paper. Hence, we leave the automatic parallelization as future work and let developers manually split the code. Note that each remoteable pointer dereferencing and the following accesses must be wrapped by a pararoutine, otherwise a compilation error occurs.

Ownership and program dependency analysis. Beehive forces programmers to follow Rust’s *ownership* constraints when developing applications, providing the compiler with a clear picture for program dependency analysis (§ 4.3). Each object in Beehive should have only one active mutable reference or multiple immutable references at any time. To modify an object, the type system ensures that the object is borrowed as mutable by one and only one remoteable reference, i.e., `RemRefMut`, from the remoteable pointer, i.e., `RemPtr`, as shown in Listing 2 (Line 16). Another mutable borrow can only occur if the previous `RemRefMut` goes out of its scope (the end of its lifetime), or explicitly releases the mutable borrow. As a result, the compiler can precisely track the flow of mutable borrowing statically, which is essential for addressing the pointer aliasing during program dependency analysis [26, 98] and data races [6], which form the basis of pararoutine generation and scheduling.

4.3 Beehive Rust Compiler

Beehive introduces a new unit of computation and scheduling, *pararoutine* (§ 4.4.1), which is derived from Rust’s experimental coroutine feature [8], an unstable language construct currently in development. The pararoutine is defined by programmers and generated by the Beehive compiler. Beehive first statically orchestrates the pararoutines to improve the con-

```

1 pararoutine! {
2     let remref_1 = remptr_1.deref_mut();
3     *remref_1 += 1;
4 };
5 pararoutine! {
6     let remref_2 = remptr_2.deref_mut();
7     *remref_2 += 1;
8 };

```

(a) User code with pararoutine interfaces

```

1 let closure_1 = #[pararoutine] || -> bool {
2     // inlined deref_mut()
3     if !present(remptr_1) {
4         post_request(remptr_1);
5         yield remptr_1;
6     }
7     let remref_1 = RemRef(remptr_1);
8     *remref_1 += 1;
9 };
10 scheduler.run_async(closure_1);
11 let closure_2 = /* ... */;
12 scheduler.run_async(closure_2);

```

(b) Compiler generated pararoutine closures

Listing 3: Code sample of pararoutine generation. The code snippet of pararoutine is created as a closure [7], e.g., `closure_1`, and passed to the scheduler.

currency according to the program dependency graph (PDG) through compiler code motion. Then, Beehive dynamically schedule the pararoutines to avoid tail latency issues during execution (§ 4.4). The pararoutine generation can be split into three basic steps: program dependency analysis, layering and optimizations, pararoutine generation and static scheduling, as shown in Figure 5.

Program dependency analysis. Beehive should guarantee that there is no data or control dependency across the concurrently executing pararoutines. Beehive compiler uses the program dependency graph (PDG) for data and dependency analysis, which is a program representation which makes the data and control dependencies explicit for each expression, and is widely employed in program analysis [37, 81, 90]. In a PDG, each vertex, e.g., u and v , corresponds to a statement, a predicate expression or a pararoutine, and an edge, $u \rightarrow v$, indicates vertex v is dependent on vertex u . A dependency edge can represent either a data dependency or a control dependency (e.g., decided by the conditional and branching statements) [10]. Dependencies between statements will prevent them from executing asynchronously. The PDG analysis relies on type safety to ensure code properties like immutability, which is ensured in the safe Rust and the APIs provided by Beehive. See Appendix 4.5 for more details.

Before constructing the PDG, Beehive applied classic loop transformations such as the unrolling, fission, and fusion [37, 85] to preprocess the code to increase potential concurrency. Additionally, Beehive provides an `async_loop` API for developers for better pararoutine static scheduling. Given that loop optimization is well-studied and not our focus, we

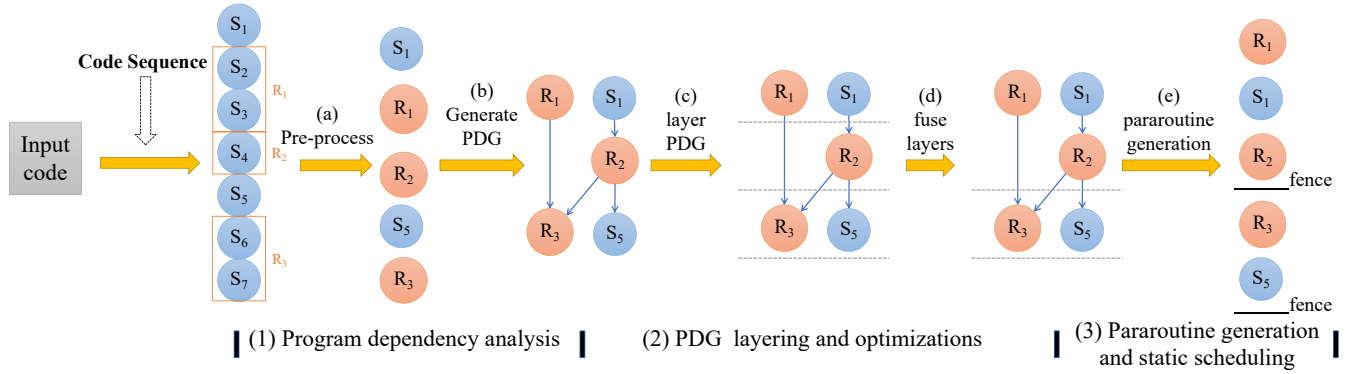


Figure 5: Automatic asynchronous code transformation. S is for statements and R is for pararoutines.

omit further details. Then, Beehive merges the statements belonging to the same pararoutine (defined by developers, as shown in Listing 3(a)) into one vertex, as shown in Figure 5 (a). Finally, Beehive creates the PDG on the optimized code sequences.

The PDG is constructed by following (Ferrante et al [37]), with pointer aliasing managed by ownership in Beehive. In particular, given two PDG vertices u, v with u preceding v in the program order, (1) a data dependency $u \rightarrow v$ exists when u, v access the same data object and at least one of them is write, and (2) a control dependency $u \rightarrow v$ exists if u determines whether or not v executes. Finally, we get a pararoutine-based PDG, as shown in Figure 5 (b).

Layering and optimizations. To enable efficient concurrent execution, the Beehive compiler groups pararoutines into *layers*. The pararoutines can execute asynchronously within each layer, while they cannot go out of the layer and must be synchronized by the fence primitives inserted between layers. Layers are initially generated by Algorithm 1. Each vertex represents a pararoutine or a statement outside the pararoutine. The compiler groups the dependency-free vertices, which have no immediate predecessors, into the same layer. Then, it removes the visited vertices from the PDG and continues the procedure until all the vertices are partitioned into different layers. Beehive applies optimizations to remove unnecessary fences by merging unnecessary layers. For example, a layer will be merged into its immediate predecessor if there is no dependency across pararoutines. This layer fusion pass is shown in Algorithm 2.

Pararoutine generation and static scheduling. The Beehive compiler statically schedules the pararoutines based on the layered PDG, as shown in Figure 5 (e). The pararoutines within a layer can execute asynchronously. Hence, Beehive can reschedule and output the pararoutines in any topology order. However, the statements and pararoutines should be scheduled by following the dependency, e.g., the vertex $S1$ and vertex $R2$ must be scheduled in sequence. In addition, the compiler tends to let the pararoutine execute

Algorithm 1 Layering the PDG

```

1: Input:  $G = \{V, E\}$  - PDG of pararoutines
2: procedure PDG-LAYERING
3:   while  $V \neq \emptyset$  do
4:     Create a new layer  $l$ 
5:     for all  $v \in V$  do
6:        $\triangleright$  Find vertices without immediate predecessors.  $\triangleleft$ 
7:       if HasNoDependency( $v$ ) then
8:          $\triangleright v$  can be either pararoutine or statement.  $\triangleleft$ 
9:         add  $v$  into  $l$ 
10:    Remove vertices in  $l$  from  $V$ 
11:  output  $l$ 

```

Algorithm 2 Layer fusion

```

1: Input:  $G = \{V, E, L\}$  - Layered PDG
2: procedure LAYER-FUSION
3:    $V_R \leftarrow \{v \in V \mid v \text{ is a pararoutine}\}$ 
4:    $l \leftarrow \text{pop the front layer from } L$ 
5:   while  $L$  is not empty do
6:      $l' \leftarrow \text{pop the front layer from } L$ 
7:     if  $\exists v_r \in l \cap V_R, \exists v \in l', v \text{ depends on } v_r$  then
8:       output  $l$ 
9:        $l \leftarrow l'$ 
10:    else
11:       $l \leftarrow l \cup l'$   $\triangleright$  fuse the two layers
12:  output  $l$ 

```

before the dependency-free statements, e.g., $R3$ and $S5$, to prepare enough instructions to fill the idle CPU during remote access triggered by the pararoutine. After the vertex sequence is scheduled, Beehive compiler inserts the fence primitives between each layer to synchronize the pararoutines.

Each pararoutine is closed by making free variables that are used but not defined in this pararoutine captured and accessible in this pararoutine [73]. The generated pararoutine is as shown in Listing 3.

4.4 Beehive Runtime

The Beehive Runtime is responsible for task scheduling and memory management.

4.4.1 Pararoutine

Beehive introduces a stackless compute abstract, *pararoutine*, which is derived from the Rust experimental coroutine [8]. A pararoutine contains a series of compute instructions and at least one remoteable pointer dereferencing (and subsequent access through remoteable references). The pararoutine supports lightweight cooperatively scheduling at the dereferencing point and can be preempted at any point with heavier overhead.

Cooperative scheduling. The efficient cooperative scheduling of pararoutines lets the Beehive programs achieve higher remote memory concurrency while consuming less CPU resources. Pararoutine’s cooperative scheduling outperforms the Shenango uthread switching by up to 2.3×, mainly due to two reasons: the reduced context and the lock-less waiting queue.

First, Pararoutines significantly reduce context switching overhead. Pararoutine switching only incurs one extra continuation invocation, which is as lightweight as a function invocation and leads to one store instruction to register continuation, as few as four instruction to restore context (*e.g.*, `mov`, `pop`), one `ret` and one indirect jump instruction. Compared to pararoutine switching, the uthread switching incurs several times more register and memory operations, including uthread state maintenance, lock operations, and register saving and restoration. For example, the Shenango uthread switching involves 30 assembly instructions, including 21 memory operations and 1 lock operation [83].

Second, exploiting concurrency within the uthread helps reduce the scheduling overhead. During the switching, only three instructions are needed to push the pararoutine into a lock-less per-thread waiting queue or pop a ready pararoutine for execution. On the contrary, the uthread needs to consider more complicated scheduling conditions, such as task stealing and shedding, which involves a series of uthread state check and locks [83]. Note that Beehive also supports task stealing at the uthread level, which works well to achieve high throughput and low latency (§ 5).

Beehive also supports thread switching when there is insufficient concurrent pararoutines within a thread. Specifically, when the program reaches a pararoutine fence, the thread will execute the scheduling code and poll the pending continuations, which may result in thread switching if it cannot acquire any ready task to process for 1 μs (an empirical threshold). Although thread switching may impair locality, we prioritize improving CPU efficiency.

Preemptive scheduling. Cooperative scheduling is good at achieving high CPU efficiency but usually fails to maintain μs-scale tail latency [70]. This is especially true because pararoutines are defined statically by developers without the

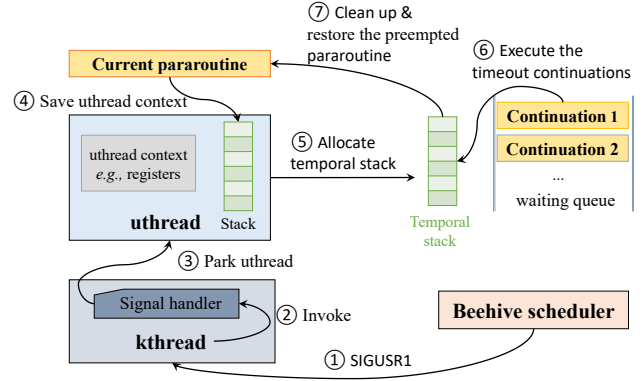


Figure 6: Preempt the large pararoutine to avoid tail latency issues

benefit of runtime information. Some pararoutines can be large and lead to long tail latency issues due to the head-of-line blocking. For example, the suspended pararoutines (continuations) can not be rescheduled to run in time due to waiting for the completion of a large pararoutine, resulting in violations of the service level objective (SLO). Hence, Beehive provides μs-scale preemption to the pararoutine to improve the predictability of latency critical applications.

The pararoutine preemption is implemented on the SIGUSR1 signal, and the procedure is shown in Figure 6. When the scheduler detects that a suspended continuation exceeds the threshold of waiting time, it sends a SIGUSR1 signal to the kthread that the uthread runs on and invokes the signal handler. As a result, the uthread that executes the pararoutine (that leads to the blocking) will be interrupted and parked. The uthread context will be saved on the stack frame. Next, the signal handler will allocate a temporal stack and pop the stranded continuations from the waiting queue to run. Finally, the scheduler will resume the suspended pararoutine on the original stack frame. The preemption consumes 1.2 μs, which is comparable to the uthread preemption [82].

4.4.2 Memory management

Beehive’s heap is divided into three spaces, as shown in Figure 7: the *native space* that resides in the local memory and is used for local-only data; the *remoteable space* that spans the local and remote memory, and the data can be swapped in and out on demand; and the *offloading space* that is used for supporting the offloaded computations.

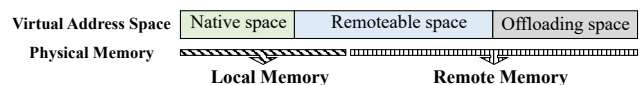


Figure 7: Memory layout of the Beehive application

Object allocation. If the programmers allocate data via the native (original) Rust data types, such as the smart pointer (`Box<T>`), the data will be put into the *native space*, which

is pinned in the local memory, and managed by the original Rust allocator.

Programmers need to explicitly use the remoteable pointer to allocate objects into the remoteable space. The objects are initially allocated in local memory and will be evicted by the *evacuator* to the remote memory when becoming cold. However, programmers can explicitly adjust the allocation site to the remote memory (for cold data) or offloading space by tuning the parameter, `position` (Listing 1). Beehive utilizes the log-structured allocator [24, 92, 93] to manage the remoteable space. Each thread maintains a thread local allocation buffer (TLAB) to reduce the lock contention. The TLAB is divided into a series of *segments*, which is set as 8MB in default by following the original design [93].

Beehive reserves a dedicated address space as the offloading space. The offloaded data is also managed by the remoteable pointer, *i.e.*, `RemPtr`, with the `offloaded` bit set. Programmers can offload computations on the offloaded data and retrieve the results via the RPC call.

Evacuator. Beehive adds a concurrent evacuator to the log-structured allocator for defragmentation and cold data eviction. When memory availability drops below the threshold (20% by default), the evacuator initiates concurrent scanning of cached local memory objects and evicts cold data to remote memory using the CLOCK replacement algorithm. This lightweight approach achieves comparable effectiveness to LRU-like alternatives [24, 111]. There are four reversed bits in remoteable pointers (Figure 4(a)) that enable optional implementation of the LRU-style algorithm.

Beehive adopts AIFM’s design, *i.e.*, *dereference scope*, to synchronize the application threads with the concurrent evacuation threads to avoid data races. However, instead of using the *dereference scope* as a coarse-grained fence, *i.e.*, the evacuator is blocked until the application threads exit their *dereference scope* at least once, Beehive uses the *dereference scope* in a per-object way similar to Atlas [24]. An object enters the *dereference scope* when being dereferenced, *e.g.*, `deref` or `deref_mut`, and exits the scope when the remoteable reference dies (going out the scope of the `RemRef` or `RemRefMut`). The benefits are twofold. First, fine-grained synchronization opportunistically facilitates the evacuator in reclaiming more space. Achieving higher remote access concurrency means that more objects will be pinned in the local memory (within the *dereference scope*), and timely memory reclamation helps alleviate the out-of-memory error due to the live-lock. Second, a frequently launched evacuator helps improve the data locality.

4.5 Limitations and Generality

Beehive relies on the Rust borrow checker for PDG analysis. Consequently, for programs with unsafe Rust code, the PDG analysis is less accurate, resulting in restricted pararoutines scheduling (§ 4.4). For example, the Rust compiler cannot properly handle the interior mutable types such as `Cell` and

`RefCell`, which are implemented with `UnsafeCell`. As a result, Beehive will give up the PDG analysis and automatic asynchronization for these affected pararoutines. For libraries with source code, safety violations can be identified by checking for unsafe code blocks [33]. In this case, Beehive can skip applying automatic asynchronization to the affected code snippets to ensure correctness. On the other hand, the code transformation should be disabled if unsafe binary libraries are used due to a lack of effective ways to detect the usage of unsafe code. In this situation, developers can still manually schedule the pararoutines through the provided Beehive APIs, *e.g.*, `async` and `await`, as conventional asynchronous programming frameworks do [27, 34, 35].

In addition, the design of Beehive is not limited to Rust borrow checker. For example, Beehive can adopt deterministic parallelization [18] and reference immutability [41, 76, 80] to perform PDG analysis and automatically transform the code into asynchronous mode in other languages. We chose to implement Beehive on Rust because it is popular and widely adopted by the system community. System developers are more familiar with Rust’s programming syntax. On the other hand, Beehive is designed to mitigate the μ s-scale hardware latency while preserving applications’ data locality. Hence, Beehive’s techniques can be potentially applied to emerging storage devices exhibiting similar I/O latencies. We leave these as future work.

5 Evaluation

5.1 Setup and Methodology

We wrote about 8,000 lines of Rust and C++ code to implement Beehive. We ran experiments with one compute server and one memory server connected by the 100 Gbps Mellanox ConnectX-5 InfiniBand adapters. Each server is equipped with 2 Intel Xeon Gold 6342 CPUs, 256 GB of memory. We use one memory server for the evaluation by default, while evaluating the scalability by using up to 24 servers in § 5.4.

Baselines. Beehive is built on Rust following the design of AIFM. Hence, we implement all the evaluated applications on AIFM with all the mentioned optimizations [92], *e.g.*, non-temporal hints and prefetching techniques, enabled for performance and latency comparison. In addition, we run original applications on two state-of-the-art kernel paging-based platform, Hermit [88] and Fastswap [11], as another baseline to evaluate the runtime overhead potentially introduced by Beehive. Both Fastswap and Hermit employ a paging-based data path. Hermit consistently outperforms Fastswap due to its higher remote access throughput by exploiting asynchrony during swap-out.

The Rust versions of applications exhibit very similar performance when compared to the C++ versions, as shown in Figure 8, under the 100% memory configuration. Hence, the performance benefits of Beehive mainly come from the good data locality and the lightweight scheduling. We imple-

mented asynchronous optimization by modifying less than 5% of the core codebase per application in the Rust versions. AIFM utilizes a kernel-bypass TCP/IP networking stack implemented by Shenango for pthreads, while Beehive, Hermit, and Fastswap employ RDMA. The TCP stack overhead in AIFM is less than 2% [92], meaning that differences in the network stacks should not impact the conclusions drawn from the evaluation.

Workloads. As shown in Table 1, we evaluated seven applications under eight workloads, including Llama2 [55, 102], an open-source large language model, Multi-Grid PDE [65], a scientific partial differential equation solver, Social Network [39], a web service framework, and a synthetic workload, Key-Value Store, that is implemented in microservice, as well as Ligra [97], a graph processing framework, LiveGraph [114], a graph database, and DataFrame [74], a real-world data analysis program.

From the perspective of memory access patterns, these benchmarks can be divided into three categories: regular, random and mixed.

First, Llama2 (LMA) and Multi-Grid PDE (MGP) are both memory-intensive with good spatial locality when processing their data in arrays and vectors. The main difference between these two applications is that LMA exhibits regular access patterns, *i.e.*, sequential patterns, when performing matrix multiplication. However, MGP exhibits mixed access patterns due to its iterative stencil computation on two grids. Llama2 is a collection of pretrained and fine-tuned large language models (LMA) optimized for dialog use cases. In the evaluation, we use the float32 model with 7 billion parameters and deploy it on the CPU side. Performing inference in CPU has been widely adopted by a series of studies [44, 62]. As mentioned in § 3, MGP uses the V-cycle multi-grid method to compute a 3D scalar Poisson equation on grids.

Second, Social Network (SN) [39] and Key-Value Store (KVS) both exhibit random memory access patterns with poor spatial data locality. SN is a web service that stores the user relationship [91], *i.e.*, a dynamic sparse graph, into multiple hash tables and allows users to retrieve, update and delete the user data, which exhibits skewed access patterns. The KVS stores its data in a huge hash map and allows clients to manipulate the data by keys under uniform distribution [4].

Third, Livegraph (LG) [114], Ligra (L) [97] and DataFrame (DF) [74] are data analytic frameworks exhibiting mixed memory access patterns. LG is a transactional graph database for real-time graph analytics. LG achieves high throughput by ensuring that adjacency list scans, a key operation in graph workloads, are purely sequential while exhibiting complicated patterns when accessing the vertices and edges. Ligra is designed for a single multicore machine and large shared memory, which is an order of magnitude faster on a per-core basis for graph traversal than distributed graph systems. The vertices and edges are stored in separate arrays. Each thread needs to access the vertices array and the edge array back and

forth, exhibiting mixed access patterns. DF is a data analytics framework in Pandas [79]. The DF organizes data as a table-structured in-memory data structure and does a series of operations, such as *slicing*, *copy* and *aggregate*, which exhibit different access patterns.

Application	Dataset
Llama2 (LMA) [102]	Llama-7B [55, 102]
Multi-Grid PDE(MGP) [65]	NAS-benchmark [65]
Social Network (SN) [39]	Facebook network-A-anon [91]
Key-Value Store (KVS)	Synthetic, 50% Get, 50% Insert [4]
Livegraph Betweenness Centrality (LG-BC) [114]	Arabic-2005 23M Vertices, 640M Edges [91]
Ligra Connected Component (L-CC) [97]	Synthetic 134M Vertices, 1B Edges [14]
DataFrame Taxi (DF-T) [74]	NYC Taxi [2]
DataFrame Flight (DF-F) [74]	Airline on-time Performance [3]

Table 1: Applications and inputs for the evaluation.

Setup. Every application is running with five different local memory configurations: 13%, 25%, 50%, 75% and 100%, each of which represents a ratio of the whole memory usage of the application that can fit in local memory. The first four configurations are used to evaluate applications' performance when using different size of remote memory. The 100% (referred to as *all local*) is used to demonstrate the runtime overhead of AIFM and Beehive, introduced by barriers, profiling and other bookkeeping overhead [92]. All applications have the same number of application and evacuation threads. The number of application threads are tuned to optimal for each application (§ 5.4).

5.2 Throughput

We first measured the throughput of the applications with varying local memory ratios. Beehive outperforms Fastswap, Hermit and AIFM, respectively, by $4.26\times$, $3.05\times$, and $1.58\times$, over the eight workloads from 13% to 75% local memory. Beehive demonstrates particularly strong performance for workloads with well-partitioned data across threads and parallelizable, independent work within threads, such as MGP and DF. These types of applications benefit significantly from Beehive's ability to maintain data locality while exploiting asynchronous execution within threads.

Llama2 (LMA) and Multi-Grid PDE (MGP). As mentioned in § 5.1, both LMA and MGP are memory-intensive applications with good spatial locality. LMA mainly performs matrix multiplication exhibiting sequential access patterns (while MGP exhibits mixed patterns due to its iterative stencil computation). The data prefetching of Fastswap, Hermit, AIFM and Beehive all work very well for LMA, resulting in little on-demand remote accesses. In addition, LMA almost has no temporal locality. Hence, LMA is not sensitive to local memory ratios under these four platforms, as shown in Figure 8(a). The performance is mainly limited by the RDMA bandwidth.

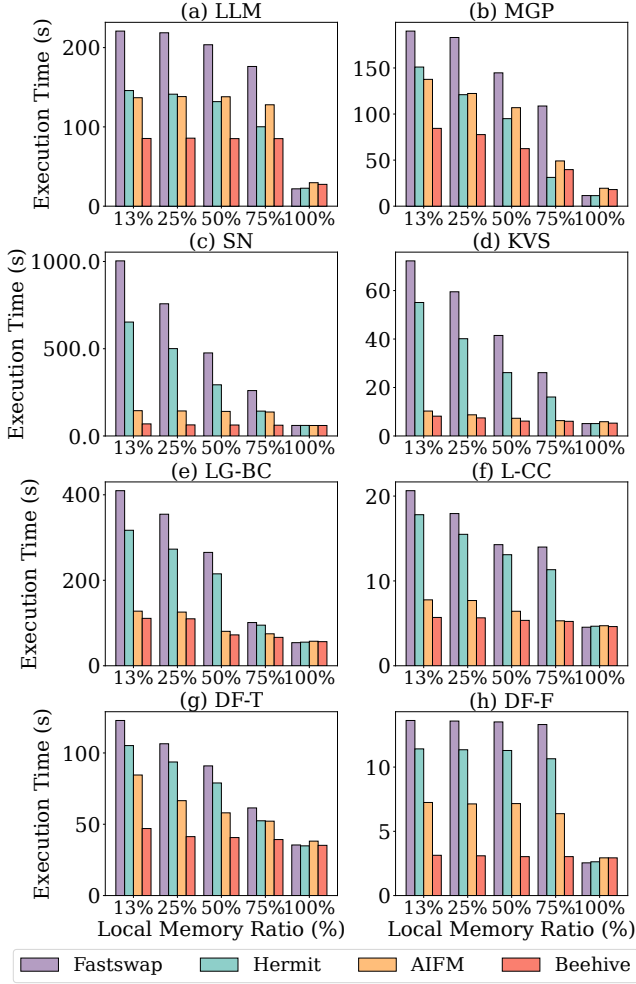


Figure 8: Throughput comparison between Fastswap, Hermit, AIFM and Beehive with varying local memory ratios.

Due to the good spatial locality of both LMA and MGP, most of fetched data will be accessed before being swapped out, which significantly alleviates the data amplification issues for the paging-based solutions, *e.g.*, Fastswap and Hermit [21, 22]. Hence, compared to Hermit, the performance improvement of runtime-based solutions, *i.e.*, AIFM and Beehive, are marginal. AIFM and Beehive outperform Fastswap by $1.56\times$ and $2.41\times$, and Hermit by $0.93\times$ and $1.47\times$ on average, due to eliminating the overhead of trapping into kernel space and rich program semantics for prefetching during the remote memory access. At the same time, as the local memory ratio increases, *e.g.*, reaching up to 75%, AIFM and Beehive even start underperforming Hermit, *e.g.*, by 36% and 21% for MGP, due to runtime overhead, such as the barrier and profiling overhead. During the process, MGP continuously updates the data, leading to a large amount of dirty data being swapped out to remote. Compared to Fastswap, the

asynchronous reclamation of Hermit [88] achieve by up to $1.94\times$ higher throughput.

However, we still observed non-trivial performance improvement by utilizing the async execution model. Beehive outperforms AIFM by $1.58\times$ and $1.54\times$ on average, correspondingly under four remote memory configurations for LMA and MGP. This is because LMA and MGP partition the data, *e.g.*, matrices and arrays, into a bunch of tiles and assign them to different threads, leading to poor data locality during thread switching. The LLC misses sharply increase as the local memory shrinks, which enlarges the thread switching frequency due to escalating remote accesses. Taking MGP as an example, on average, AIFM introduces $3.5\times$, $5.3\times$, $5.5\times$ and $7.3\times$ more LLC misses than Beehive under 75%, 50%, 25% and 13% local memory ratios, correspondingly.

Social Network (SN) and Key-Value Store (KVS). SN exhibits random memory access with highly skewed distribution, while KVS exhibits uniformed memory access pattern. As a result, under the four remote memory configuration, both Beehive and AIFM significantly outperform Fastswap by $8.27\times$ and $5.14\times$, and Hermit by $5.4\times$ and $1.7\times$ for SN and KVS on average, respectively. Beehive and AIFM fetch data in fine-grained object granularity, significantly reducing the data amplification seen in Fastswap and Hermit.

Unlike in LMA and MGP, the data of KVS is shared by application uthtreads. Each uthread scan the data randomly so that the uthread does not have its own hotspot. Hence, even the uthread switching number reaches up to 2.3M/s under 13% local memory, Beehive’s LLC misses is only slightly $2.6\times$ less than AIFM. However, Beehive’s scheduling overhead is $6.9\times$ smaller than AIFM, leading to $1.28\times$ higher throughput. As discussed in § 4.4, although Beehive has the same number of application uthtreads, the number of active uthtreads is much less than in AIFM. In addition, the pararoutine switching is $2.3\times$ faster than uthtreads. Due to the skewed workload of SN, the number of remote access is significantly reduced. However, under the four remote memory configurations, Beehive still outperforms AIFM by $2.21\times$ on average.

Livegraph BC (LG-BC) and Ligra CC (L-CC). Due to the mixed access patterns, both Livegraph and Ligra have I/O amplification issues. Hence, compared to Fastswap and Hermit, Beehive improves the performance by $3.03\times$ and $2.44\times$ for LG-BC, and $3.04\times$ and $2.62\times$ for L-CC, correspondingly. AIFM improves the performance by $2.67\times$, $2.15\times$ and $2.46\times$, $2.12\times$ for L-BC and L-CC, correspondingly.

Compared to AIFM, Beehive’s performance improvement is modest, by $1.13\times$ and $1.23\times$ for LG-BC and L-CC, respectively, on average. The performance benefits of Beehive are limited mainly due to two reasons: first, the complicated graph results in intricate PDG, which limits the number of generated pararoutines in each layer (*e.g.*, 3.8 pararoutines per layer on average for the L-CC); second, the LG-BC is very memory-intensive: the average RDMA bandwidth reaches up

to 8.7GB/s under the 13% local memory. Both Beehive and AIFM are quickly limited by the limited network bandwidth.

DataFrame Taxi (DF-T) and Flight (DF-F). We perform two sets of operations on two different datasets. For the DF-T, a series of *Filter*, *Range extraction*, *GroupBy* operations are performed on the NYC Taxi dataset [2], while *Aggregate* and *Copy* are performed on the Flight data set [3]. Offloading is disabled in Beehive and AIFM to evaluate the efficiency of remote memory access. Compared to Fastswap and Hermit, the performance improvements of AIFM is less significant for the DF-T ($1.46\times$ and $1.25\times$ on average) than DF-F ($1.94\times$ and $1.6\times$ on average). This is mainly because DF-F contains more fine-grained data access.

For DF-T and DF-F, Beehive outperforms AIFM by $1.54\times$ and $2.27\times$ on average, under the four remote memory configurations. The performance improvements mainly come from the reduced LLC misses. As aforementioned, DataFrame organizes data as a table-structured in-memory data structure that contains millions of columns and rows. The columns are sliced into chunks and evenly assigned to different utthreads for load-balancing and high parallelism. For AIFM, frequent utthread switching leads to frequent changes of thread local data, resulting in $2.8\times$ and $8.8\times$ higher LLC misses than Beehive, for DF-T and DF-F, respectively.

5.3 Latency

We then evaluated the latency of the applications with a fixed configuration of 16 cores and 13% local memory ratio. We choose Social Network (SN) and Key-Value Store (KVS) to measure the tail latency of different platforms under varied offered load.

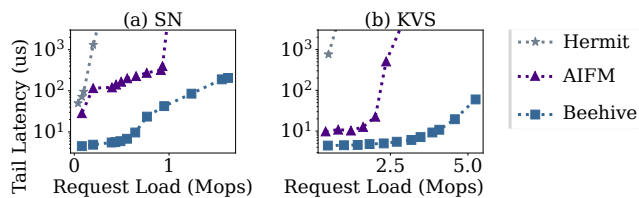


Figure 9: 90th latency as a function of offered load

Social Network (SN). Both SN and KVS exhibit random access patterns, leading to non-trivial I/O amplification issues for Hermit. Hence, the 90th tail latency of Hermit rapidly grows under the request rate of 0.1 Mops and 0.5 Mops for SN and KVS, respectively. Compared to AIFM, Beehive improves the compute capacity of cores by reducing the LLC misses (by 40%) and scheduling overhead (by 80%), significantly reducing the queuing latency of SN. As a result, Beehive's tail latency stays low until 0.64 Mops, and then the tail latency rapidly grows.

Key-Val Store (KVS). KVS is similar to SN as they both access key-value pairs from hash tables. The difference is that the request key in KVS follows a uniform distribution,

as opposed to highly skewed accessing in SN. For the 90th latency, Beehive outperforms AIFM and Hermit by $5.5\times$ and $2,869\times$ under the load of 2Mops. Beehive achieves superior efficiency in two aspects—(1) improved CPU efficiency due to reduced WSS and (2) the effective preemptive scheduling.

5.4 Performance Drill Down

Sensitivity to the number of threads. As demonstrated in Figure 10, at first, the performance of four kinds of applications running on AIFM all increases with the number of utthreads due to improving CPU efficiency by switching to other ready tasks after issuing the requests, effectively hiding the μ s-scale far memory access latency. However, as the number of utthreads increases, applications' throughput decreases to varying degrees due to LLC misses and scheduling overhead, exhibiting different optimal numbers of utthreads for AIFM.

On the contrary, Beehive is not sensitive to the number of utthreads. The performance differences are within 4% for the evaluated applications. This is due to Beehive's cooperative scheduling approach, where each utthread only yields after completing its task, effectively maintaining only one active utthread per core. L-CC-Beehive exhibits the most significant performance degradation. This is because the Beehive compiler cannot generate enough pararoutines for each layer, eventually resulting in utthread switching (§ 5.2).

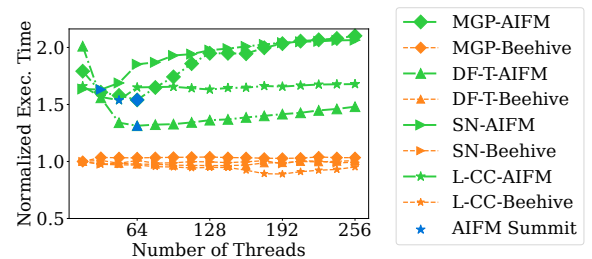


Figure 10: 16 cores are used, with 13% local memory ratio. The execution time is normalized to Beehive with 16 application utthreads.

LLC miss and scheduling overhead. In this section, we evaluate the LLC miss and scheduling overhead between Hermit and Beehive for four kinds of applications, as shown in Figure 11. Compared to AIFM, Beehive reduced LLC misses by 57% and scheduling overhead by 82%, on average.

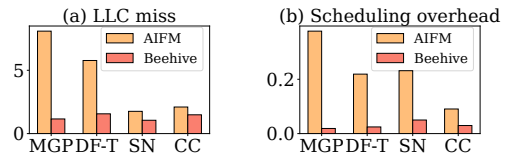


Figure 11: LLC miss (normalized to Hermit) and scheduling overhead to the CPU time, under 13% local memory ratio.

Scalability of the memory servers. Developing memory-disaggregated applications based on μ s-scale I/O fabrics, e.g.,

InfiniBand [1], has two main benefits—first, it can utilize the conventional hardware; second, it can build a memory pool larger than rack-scale. Hence, we evaluate the scalability of Beehive by requiring memory resources from up to 24 memory servers (connected to one switch). As shown in Figure 12, performance degradation is limited to 2.5% for all the applications. However, we observed significant performance variation for the SN. The root cause is that SN exhibits random memory access patterns with a large memory footprint, resulting in frequent and unpredictable queue pair (QP) switching between different servers. According to previous research [34, 103], excessive QP switching can potentially lead to performance fluctuation.

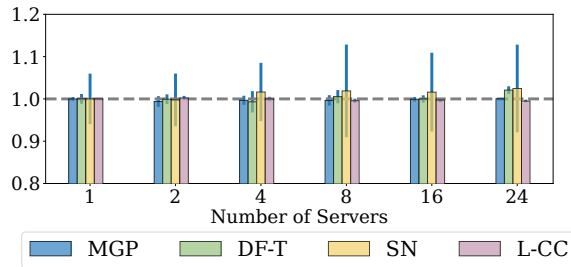


Figure 12: Impact of the number of memory servers on performance, with error bars, normalized to using one memory server

6 Related Work

Emerging hardware. Hardware-based schemes, such as the Pond [60] and Kona [21], are expected to provide direct load/store access to remote memory in cache line granularity, sub-microsecond-level data accessing latency, and hardware-support cache coherency across servers. However, these schemes need specialized hardware that is not commercially available at this moment. In addition, they still need software support to mitigate the μ s-scale latency due to inefficient concurrency of remote accesses [15, 57]. For example, the number of miss status holding registers (MSHRs, used by CPUs to track the on-the-fly memory requests) is so limited that it cannot provide enough memory parallelism to hide nanosecond-scale local memory (DRAM) access [57, 104].

Kernel paging. Kernel-based schemes are popular because they are transparent to existing applications. The remote memory works as the second-tier memory layer and is managed by the kernel paging system—the cold pages will be swapped out to remote memory when there is not enough local memory space, and the swapped-out pages will be fetched back when being accessed by the application. However, the kernel-based schemes are notorious for non-trivial page fault overhead and I/O amplification [22, 24, 29, 89, 92] that not only degrades the throughput but also leads to performance variation and latency issues [88, 108].

Prefetching. Prefetching is effective to hide the data access latency. The prefetching techniques have been studied exten-

sively, and a series of software-based [68, 71, 92, 108] and hardware-based [50, 51, 61, 96, 113] prefetchers are proposed. Beehive can also benefit from these works and adopts similar semantics-guided prefetching techniques as AIFM [92].

Disaggregated memory frameworks. A series of frameworks [20, 23, 34, 35, 53, 54, 67, 75, 77, 78, 99, 103, 106, 107] have been proposed for distributed memory models, such as distributed shared memory (DSM) [63] and disaggregated memory [95]. Although some of these works adopt asynchronous programming models, they mainly focus on providing high-performance asynchronous remote access primitives [34, 35, 78, 103] or reducing the synchronization overhead while maintaining the data coherency across distributed servers [20, 67, 106]. The developers are still expected to handle the asynchronous remote access manually.

For example, Storm [78] proposes *one-two-sided* operations, which use one-sided reads for simple data retrieval and two-sided RPCs for complicated memory operations, such as doing pointer chasing in a tree structure. In addition, FaRM [34] provides an event-driven programming model and still asks developers to define and register the *continuations* explicitly, which Beehive generates them automatically. For DSM frameworks, GAM [20] adds an extra level of cache atop the distributed memory to reduce costly remote memory access by exploiting applications’ data locality, while dRust [67] reduces RDMA communications further during the cache coherency maintenance by leveraging the ownership semantics. In addition, RackOut [77] re-organizes the data in rack-scale granularity to improve the load balance of pooled memory. These works are orthogonal to Beehive, which mainly focuses on improving the data locality and reducing the scheduling overhead for applications exhibiting excessive thread switching. Their techniques can be potentially integrated into Beehive to enhance the network performance. We leave this to future work.

7 Conclusion

We present Beehive, a runtime framework developed for memory-disaggregated platforms. Beehive achieves superior remote access concurrency by exploiting the asynchrony within each thread while keeping the programming usability as the multithreaded synchronous programming framework.

Acknowledgment

We thank the reviewers for their comments and are particularly grateful to our shepherd Malte Schwarzkopf for his feedback. This work is supported by National Key Research and Development Program of China under grant 2024YFE0204100, the National Natural Science Foundation of China (62090024, 62232015, 62302479, U23B2020) and the Innovation Funding of ICT, CAS (E361010, E261110). Dr. Jie Zhang is supported in part by the Natural Science Foundation of China under Grant No. 62332021 and 62472007.

A Appendix of Beehive Programming

A.1 Case Study for Programming Effort

To demonstrate Beehive’s programming efficiency, we have refactored and minimized a representative code snippet from the Social Network (SN, Table 1) application to construct a minimal working example. Through careful abstraction of non-essential implementation details, we have distilled the core functionality while preserving the essential characteristics of the original code snippet. Next, we will use the function `post_reply`, which is used for posting a reply from user A, *i.e.*, `sender`, to user B, *i.e.*, `receiver`, as an example to demonstrate the programming effort of Beehive when compared to that of AIFM.

Let us start from implementing the function in AIFM, as shown in Listing 4. When the `sender` replies to the `receiver`, the `post_reply` function accesses their profiles stored in remote objects to generate a textual message “`sender name replies to receiver name`”. A dereference scope is created in Line 2 to synchronize the application threads with the evacuator in the specific scope, *i.e.*, from Line 2 to Line 14, to prevent potential data races (Section 4.4.2). The evacuator will be blocked when the remote objects, such as the object referenced by `sender` in Line 7, are being dereferenced and accessed. Lines 5-6 retrieve `sender_ptr`, the remoteable pointer referencing the `sender`’s profile data. This pointer is then dereferenced to fetch the corresponding data from the remote in Line 7. The value of the filed `name` of `sender` is copied in Line 8 after the remote access is complete. Lines 9-12 perform analogous operations to obtain `receiver`’s name. Finally, the message is composed of data retrieved from the remoteable objects.

```
1 std::string post_reply(const Request& req) {
2     DerefScope scope;
3     std::string sender_name;
4     std::string receiver_name;
5     RemUniquePtr<User>& sender_ptr =
6         get_user(req.sender_id);
7     const User* sender = sender_ptr.deref(scope);
8     sender_name = sender->name;
9     RemUniquePtr<User>& receiver_ptr =
10        get_user(req.receiver_id);
11    const User* receiver = receiver_ptr.deref(scope);
12    receiver_name = receiver->name;
13    return sender_name + " replies " + receiver_name;
14 }
```

Listing 4: Programming with the AIFM.

The code in Listing 5 is basically the same as those written with AIFM, except for some annotations. To enable asynchronous execution in Beehive, developers simply annotate the two remote access operations with `pararoutine!` (Line 4, Line 9), and the compiler will generate the `pararoutines` as discussed in § 4.3. In contrast, manual implementation using Rust coroutines require significant additional code, including yield condition checks, waiting for coroutines to complete

execution, and even manually scheduling and resuming the suspended coroutines on remote access completion. Thus manual approach substantially may increase the development effort, resulting in potential concurrent bugs and performance issues [9, 15, 56, 105].

```
1 fn post_reply(req: &Request) -> String {
2     let mut sender_name = String::new();
3     let mut receiver_name = String::new();
4     pararoutine! {
5         let sender_ptr = get_user(req.sender_id);
6         let sender = sender_ptr.deref();
7         sender_name = sender.name as String;
8     }
9     pararoutine! {
10        let receiver_ptr = get_user(req.receiver_id);
11        let receiver = receiver_ptr.deref();
12        receiver_name = receiver.name as String;
13    }
14    format!("{}", replies {}, sender_name,
15           receiver_name)
15 }
```

Listing 5: Programming with the Beehive.

A.2 Performance Impact of Beehive Asynchronization

Figure 13 demonstrates the performance differences with (*Beehive-async*) and without (*Beehive-sync*) compiler code transformation, which is responsible for generating and enabling the synchronous execution of `pararoutines`. For the code written in Beehive while disabling the code transformation, the annotated `pararoutines` will be ignored, and the code will execute in the conventional multithreaded synchronous model—switching to another user-level thread when encountering remote accesses (§ 4.4). As a result, the performance of MGP, DF-T, SN, and L-CC on average drops by 72.6%, 70.9%, 81.1% and 32.8%, correspondingly, when compared to enabling the code transformation, *i.e.*, *Beehive-async*. However, *Beehive-sync* still outperforms AIFM by 1.09× on average due to the optimized evacuator and the adoption of atomic instructions (§ 4.4).

A.3 Preemptive Scheduling

Improvement from coroutine. `Pararoutines` are fundamentally based on stackless coroutines but are automatically scheduled by the compiler and runtime system. This automation eliminates the need for explicit `await` statements, streamlining the programming process. The data structure of `pararoutines` is designed with additional fields that support runtime scheduling and resumption (e.g., the pointer of the accessing remoteable object), enhancing operational efficiency. A significant improvement over traditional coroutines is the support for preemptive scheduling.

In addition, a cache-locality-aware allocator is implemented for `pararoutine` frame allocation to alleviate the context switching overhead further, drawing inspiration from the slab [19] and the TCMalloc [40] allocators. Each allocation is lock-free, requiring less than 10 instructions, with all frames

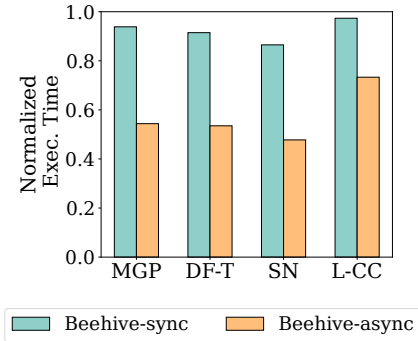


Figure 13: Performance comparison with (*i.e.*, Beehive-async) and without (*i.e.*, Beehive-sync) asynchronous remote access by enabling and disabling the Rust compiler code transformation. The execution time is normalized to AIFM. All experiments are executed under 13% local memory ratio.

allocated from a thread-private memory block that fits in the L1 cache to minimize cache misses.

Preemption. The Beehive preemption is designed to alleviate the HoL (head-of-line) blocking within latency-critical applications exhibiting wide service time distribution [32, 52]. For example, SN contains different kinds of requests whose service time, *i.e.*, execution time without queuing latency, ranges from $0.7\mu s$ to $76\mu s$, a tenfold difference, as shown in Figure 14. As a result, the heavy-tailed request, *i.e.*, *Compose post*, can easily block hundreds of short requests and enlarge their execution time by more than $200\times$, hurting the overall responsiveness. This is because, when compared to short requests, heavy-tailed requests usually contain much larger and more pararoutines, which can occupy excessive CPU resources under the cooperative scheduling. In this section, we set an execution threshold, *i.e.*, $14\mu s$ (20 times the average execution time), for the short requests to demonstrate how preemptive scheduling help improve the SLOs of SN. Note that the service time of KVS requests, *i.e.*, *Get* and *Insert*, that are used in this paper are both short requests, as shown in Figure 14. Hence, we only evaluate the SN in this section.

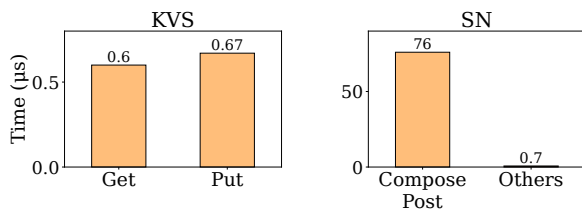


Figure 14: The service time of requests used in SN and KVS.

We tested the effects of preemptive scheduling and cooperative scheduling on request latency under a local memory ratio of 13% and a load of 0.8 Mops to avoid lacking computing capability. As illustrated in Figure 15, preemptive scheduling significantly reduces the occurrence of short re-

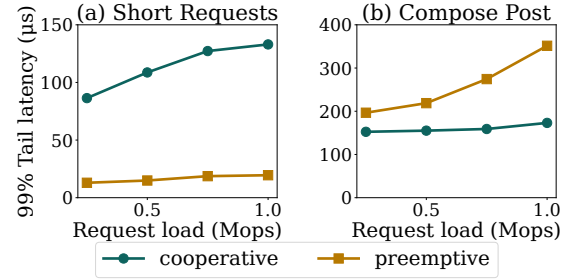


Figure 15: The 99th tail latency of SN requests under 13% local memory.

quest 99th latencies exceeding $20\mu s$. The tail latency is 85% smaller than that of cooperative scheduling on average. At the same time, the 99th latency of the heavy-tailed request is slightly enlarged by 30% on average when compared to that of cooperative scheduling. Note that we add a preemption mechanism to the stackless pararoutine without fine-tuning its scheduling policy, which is not the main contribution of this work. The previous μs -scale scheduling policies, such as the Concord [49], Perséphone [32] and Shinjuku [52] are orthogonal to Beehive. We will improve the scheduling efficiency according to previous research in future work.

References

- [1] *White paper: Introduction to InfiniBand*. Mellanox Technologies Inc., 2003.
- [2] Tlc trip record data. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>, 2016.
- [3] Airline on-time performance data. <https://www.kaggle.com/datasets/bulter22/airline-data>, 2021.
- [4] Ycsb - yahoo! cloud serving benchmark. <https://github.com/brianfrankcooper/YCSB/>, 2021.
- [5] C++ coroutines. <https://en.cppreference.com/w/cpp/language/coroutines>, 2023.
- [6] Data races and race conditions in rust. <https://doc.rust-lang.org/nomicon/races.html>, 2023.
- [7] Rust closures. <https://doc.rust-lang.org/rust-by-example/fn/closures.html>, 2023.
- [8] The rust unstable book. <https://doc.rust-lang.org/beta/unstable-book/language-features/coroutines.html>, 2023.

- [9] A. Adya, J. Howell, M. Theimer, B. Bolosky, and J. Douceur. Cooperative task management without manual stack management. In *2002 USENIX Annual Technical Conference (USENIX ATC 02)*, Monterey, CA, June 2002. USENIX Association.
- [10] F. E. Allen. Control flow analysis. *SIGPLAN Not.*, 5(7):1–19, jul 1970.
- [11] E. Amaro, C. Branner-Augmon, Z. Luo, A. Ousterhout, M. K. Aguilera, A. Panda, S. Ratnasamy, and S. Shenker. Can far memory improve job throughput. In *EuroSys*, 2020.
- [12] N. Amit. Optimizing the TLB shutdown algorithm with page access tracking. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, pages 27–39, Santa Clara, CA, July 2017. USENIX Association.
- [13] N. Amit, A. Tai, and M. Wei. Don’t shoot down tlb shutdowns! *EuroSys ’20*, 2020.
- [14] D. Anderson, G. E. Blelloch, L. Dhulipala, M. Dobson, and Y. Sun. The problem-based benchmark suite (pbbs), v2. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP ’22*, page 445–447, New York, NY, USA, 2022. Association for Computing Machinery.
- [15] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. Attack of the killer microseconds. *Commun. ACM*, 60(4):48–54, 2017.
- [16] A. Belay, G. Prekas, A. Klimovic, S. Grossman, C. Kozyrakis, and E. Bugnion. IX: A protected data-plane operating system for high throughput and low latency. In *OSDI*, pages 49–65, 2014.
- [17] B. Berg, D. S. Berger, S. McAllister, I. Grosz, S. Gunasekar, J. Lu, M. Uhlar, J. Carrig, N. Beckmann, M. Harchol-Balter, and G. R. Ganger. The Cache-Lib caching engine: Design and experiences at scale. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 753–768. USENIX Association, Nov. 2020.
- [18] R. L. Bocchino, V. S. Adve, D. Dig, S. V. Adve, S. Heumann, R. Komuravelli, J. Overbey, P. Simmons, H. Sung, and M. Vakilian. A type and effect system for deterministic parallel java. In *Proceedings of the 24th ACM SIGPLAN Conference on Object Oriented Programming Systems Languages and Applications, OOPSLA ’09*, page 97–116, New York, NY, USA, 2009. Association for Computing Machinery.
- [19] J. Bonwick and J. Adams. Magazines and vmem: Extending the slab allocator to many CPUs and arbitrary resources. In *2001 USENIX Annual Technical Conference (USENIX ATC 01)*, Boston, MA, June 2001. USENIX Association.
- [20] Q. Cai, W. Guo, H. Zhang, D. Agrawal, G. Chen, B. C. Ooi, K.-L. Tan, Y. M. Teo, and S. Wang. Efficient distributed memory management with rdma and caching. *Proc. VLDB Endow.*, 11(11):1604–1617, July 2018.
- [21] I. Calciu, M. T. Imran, I. Puddu, S. Kashyap, H. A. Maruf, O. Mutlu, and A. Kolli. Rethinking software runtimes for disaggregated memory. In *ASPLOS*, pages 79–92, 2021.
- [22] I. Calciu, I. Puddu, A. Kolli, A. Nowatzky, J. Gandhi, O. Mutlu, and P. Subrahmanyam. Project pberry: Fpga acceleration for remote memory. *HotOS ’19*, pages 127–135, New York, NY, USA, 2019. Association for Computing Machinery.
- [23] J. B. Carter, J. K. Bennett, and W. Zwaenepoel. Implementation and performance of munin. In *Proceedings of the Thirteenth ACM Symposium on Operating Systems Principles, SOSP ’91*, page 152–164, New York, NY, USA, 1991. Association for Computing Machinery.
- [24] L. Chen, S. Liu, C. Wang, H. Ma, Y. Qiao, Z. Wang, C. Wu, Y. Lu, X. Feng, H. Cui, S. Lu, and G. H. Xu. A tale of two paths: Toward a hybrid data plane for efficient far-memory applications. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [25] D. Clarke, J. Östlund, I. Sergey, and T. Wrigstad. *Ownership Types: A Survey*, pages 15–58. Springer Berlin Heidelberg, Berlin, Heidelberg, 2013.
- [26] D. G. Clarke, J. M. Potter, and J. Noble. Ownership types for flexible alias protection. In *Proceedings of the 13th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA ’98*, page 48–64, New York, NY, USA, 1998. Association for Computing Machinery.
- [27] T. Contributors. Tokio - an asynchronous rust runtime. <https://tokio.rs/>, 2024.
- [28] A. Craik and W. Kelly. Using ownership to reason about inherent parallelism in object-oriented programs. In R. Gupta, editor, *Compiler Construction*, pages 145–164, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
- [29] S. Cui, L. Jin, K. Nguyen, and C. Wang. Semswap: semantics-aware swapping in memory disaggregated datacenters. In *Proceedings of the 13th ACM SIGOPS Asia-Pacific Workshop on Systems, APSys ’22*, page

- 9–17, New York, NY, USA, 2022. Association for Computing Machinery.
- [30] F. Dabek, N. Zeldovich, F. Kaashoek, D. Mazières, and R. Morris. Event-driven programming for robust software. In *Proceedings of the 10th Workshop on ACM SIGOPS European Workshop, EW 10*, page 186–189, New York, NY, USA, 2002. Association for Computing Machinery.
 - [31] M. Das, S. Lerner, and M. Seigle. Esp: path-sensitive program verification in polynomial time. In *Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation, PLDI '02*, page 57–68, New York, NY, USA, 2002. Association for Computing Machinery.
 - [32] M. Demoulin, J. Fried, I. Pedisich, M. Kogias, B. T. Loo, L. T. X. Phan, and I. Zhang. When idling is ideal: Optimizing tail-latency for highly-dispersed datacenter workloads with perséphone. In *SOSP 2021*, October 2021.
 - [33] R. P. Developers. Unsafe rust - the rust programming language. <https://doc.rust-lang.org/book/ch20-01-unsafe-rust.html>, 2025.
 - [34] A. Dragojević, D. Narayanan, M. Castro, and O. Hodson. FaRM: Fast remote memory. In *NSDI*, pages 401–414, 2014.
 - [35] A. Dragojević, D. Narayanan, E. B. Nightingale, M. Renzelmann, A. Shamis, A. Badam, and M. Castro. No compromises: distributed transactions with consistency, availability, and performance. In *Proceedings of the 25th symposium on operating systems principles*, pages 54–70, 2015.
 - [36] Y. Feng and E. D. Berger. A locality-improving dynamic memory allocator. In *Proceedings of the 2005 Workshop on Memory System Performance, MSP '05*, page 68–77, New York, NY, USA, 2005. Association for Computing Machinery.
 - [37] J. Ferrante, K. J. Ottenstein, and J. D. Warren. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 9(3):319–349, 1987.
 - [38] S. J. Fink, E. Yahav, N. Dor, G. Ramalingam, and E. Geay. Effective typestate verification in the presence of aliasing. *ACM Trans. Softw. Eng. Methodol.*, 17(2), may 2008.
 - [39] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, K. Hu, M. Pancholi, Y. He, B. Clancy, C. Colen, F. Wen, C. Leung, S. Wang, L. Zaruvinisky, M. Espinosa, R. Lin, Z. Liu, J. Padilla, and C. Delimitrou. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '19*, page 3–18, New York, NY, USA, 2019. Association for Computing Machinery.
 - [40] Google. Tcmalloc overview. <https://google.github.io/tcmalloc/overview.html>, 2023.
 - [41] C. S. Gordon, M. J. Parkinson, J. Parsons, A. Bromfield, and J. Duffy. Uniqueness and reference immutability for safe parallelism. In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications, OOPSLA '12*, page 21–40, New York, NY, USA, 2012. Association for Computing Machinery.
 - [42] D. Gouk, S. Lee, M. Kwon, and M. Jung. Direct access, High-Performance memory disaggregation with DirectCXL. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 287–294, Carlsbad, CA, July 2022. USENIX Association.
 - [43] J. Gu, Y. Lee, Y. Zhang, M. Chowdhury, and K. G. Shin. Efficient memory disaggregation with infiniswap. In *NSDI*, pages 649–667, 2017.
 - [44] Z. Guo, Z. He, and Y. Zhang. Mira: A program-behavior-guided far memory system. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, page 692–708, New York, NY, USA, 2023. Association for Computing Machinery.
 - [45] Z. Guo, Y. Shan, X. Luo, Y. Huang, and Y. Zhang. Clio: A hardware-software co-designed disaggregated memory system. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '22*, pages 417–433, New York, NY, USA, 2022. Association for Computing Machinery.
 - [46] B. Hardekopf and C. Lin. The ant and the grasshopper: fast and accurate pointer analysis for millions of lines of code. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '07*, page 290–299, New York, NY, USA, 2007. Association for Computing Machinery.
 - [47] C. Hu, C. Wang, S. Wang, N. Sun, Y. Bao, J. Zhao, S. Kashyap, P. Zuo, X. Chen, L. Xu, Q. Zhang, H. Feng, and Y. Shan. Skadi: Building a distributed runtime for

- data systems in disaggregated data centers. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems*, HOTOS '23, page 94–102, New York, NY, USA, 2023. Association for Computing Machinery.
- [48] J. T. Humphries, K. Kaffes, D. Mazières, and C. Kozyrakis. A case against (most) context switches. In *Proceedings of the Workshop on Hot Topics in Operating Systems*, HotOS '21, page 17–25, New York, NY, USA, 2021. Association for Computing Machinery.
- [49] R. Iyer, M. Unal, M. Kogias, and G. Candea. Achieving microsecond-scale tail latency efficiently with approximate optimal scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 466–481, 2023.
- [50] A. Jain and C. Lin. Linearizing irregular memory accesses for improved correlated prefetching. In *2013 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2013.
- [51] D. Joseph and D. Grunwald. Prefetching using markov predictors. *SIGARCH Comput. Archit. News*, 25(2):252–263, may 1997.
- [52] K. Kaffes, T. Chong, J. T. Humphries, A. Belay, D. Mazières, and C. Kozyrakis. Shinjuku: Pre-emptive scheduling for second-scale tail latency. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 345–360, Boston, MA, Feb. 2019. USENIX Association.
- [53] A. Kalia, M. Kaminsky, and D. Andersen. Datacenter RPCs can be general and fast. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 1–16, Boston, MA, Feb. 2019. USENIX Association.
- [54] A. Kalia, M. Kaminsky, and D. G. Andersen. FaSST: Fast, scalable and simple distributed transactions with two-sided (RDMA) datagram RPCs. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 185–201, 2016.
- [55] A. Karpathy. llama2.c. <https://github.com/karpathy/llama2.c>, 2024.
- [56] M. Karsten and S. Barghi. User-level threading: Have your cake and eat it too. *Proc. ACM Meas. Anal. Comput. Syst.*, 4(1), may 2020.
- [57] Y. Kim, H. Kim, and W. J. Song. Nomad: Enabling non-blocking os-managed dram cache via tag-data decoupling. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 193–205, 2023.
- [58] D. Lea. Dlmalloc: A memory allocator. <https://gee.cs.oswego.edu/dl/html/malloc.html>, 1996.
- [59] D. Leijen, B. Zorn, and L. de Moura. Mimalloc: Free list sharding in action. In A. W. Lin, editor, *Programming Languages and Systems*, pages 244–265, Cham, 2019. Springer International Publishing.
- [60] H. Li, D. S. Berger, L. Hsu, D. Ernst, P. Zardoshti, S. Novakovic, M. Shah, S. Rajadnya, S. Lee, I. Agarwal, M. D. Hill, M. Fountoura, and R. Bianchini. Pond: Cxl-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023*, pages 574–587, New York, NY, USA, 2023. Association for Computing Machinery.
- [61] H. Li, K. Liu, T. Liang, Z. Li, T. Lu, H. Yuan, Y. Xia, Y. Bao, M. Chen, and Y. Shan. Hopp: Hardware-software co-designed page prefetching for disaggregated memory. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 1168–1181, 2023.
- [62] J. Li, L. Zhao, Y. Yang, K. Zhan, and K. Li. Tetris: Memory-efficient serverless inference through tensor sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, Carlsbad, CA, July 2022. USENIX Association.
- [63] K. Li and P. Hudak. Memory coherence in shared virtual memory systems. *ACM Trans. Comput. Syst.*, 7(4):321–359, Nov. 1989.
- [64] S. Li, K. Chen, J. B. Brockman, and N. P. Jouppi. *Performance Impacts of Non-blocking Caches in Out-of-order Processors*. HP Laboratories, 2011.
- [65] J. Löff, D. Griebler, G. Mencagli, G. A. de Araujo, M. Torquati, M. Danelutto, and L. G. Fernandes. The NAS parallel benchmarks for evaluating C++ parallel programming frameworks on shared-memory architectures. *Future Gener. Comput. Syst.*, 125:743–757, 2021.
- [66] H. Ma, S. Liu, C. Wang, Y. Qiao, M. D. Bond, S. M. Blackburn, M. Kim, and G. H. Xu. Mako: A low-pause, high-throughput evacuating collector for memory-disaggregated datacenters. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2022*, pages 92–107, 2022.

- [67] H. Ma, Y. Qiao, S. Liu, S. Yu, Y. Ni, Q. Lu, J. Wu, Y. Zhang, M. Kim, and H. Xu. DRust: Language-Guided distributed shared memory with fine granularity, full transparency, and ultra efficiency. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 97–115, Santa Clara, CA, July 2024. USENIX Association.
- [68] H. A. Maruf and M. Chowdhury. Effectively prefetching remote memory with Leap. In *USENIX ATC*, pages 843–857, 2020.
- [69] N. D. Matsakis and F. S. Klock. The rust language. In *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology, HILT '14*, page 103–104, New York, NY, USA, 2014. Association for Computing Machinery.
- [70] S. McClure, A. Ousterhout, S. Shenker, and S. Ratnasamy. Efficient scheduling policies for Microsecond-Scale tasks. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 1–18, Renton, WA, Apr. 2022. USENIX Association.
- [71] M. K. McKusick, W. N. Joy, S. J. Leffler, and R. S. Fabry. A fast file system for unix. *ACM Trans. Comput. Syst.*, 2(3):181–197, aug 1984.
- [72] J. Meng, J. W. Sheaffer, and K. Skadron. Exploiting inter-thread temporal locality for chip multithreading. In *2010 IEEE International Symposium on Parallel and Distributed Processing (IPDPS)*, pages 1–12, 2010.
- [73] Y. Minamide, G. Morrisett, and R. Harper. Typed closure conversion. In *Proceedings of the 23rd ACM SIGPLAN-SIGACT symposium on principles of programming languages*, pages 271–283, 1996.
- [74] H. Moein. C++ dataframe for statistical, financial, and ml analysis. <https://github.com/hosseini/moein/DataFrame>, 2020.
- [75] J. Nelson, B. Holt, B. Myers, P. Briggs, L. Ceze, S. Kahan, and M. Oskin. Latency-Tolerant software distributed shared memory. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, pages 291–305, Santa Clara, CA, July 2015. USENIX Association.
- [76] J. Noble, D. Clarke, and J. Potter. Object ownership for dynamic alias protection. In *Proceedings Technology of Object-Oriented Languages and Systems. TOOLS 32*, pages 176–187, 1999.
- [77] S. Novakovic, A. Daglis, E. Bugnion, B. Falsafi, and B. Grot. The case for rackout: Scalable data serving using rack-scale systems. In *SoCC*, pages 182–195, 2016.
- [78] S. Novakovic, Y. Shan, A. Kolli, M. Cui, Y. Zhang, H. Eran, B. Pismenny, L. Liss, M. Wei, D. Tsafrir, and M. Aguilera. Storm: a fast transactional dataplane for remote data structures. In *Proceedings of the 12th ACM International Conference on Systems and Storage, SYSTOR '19*, page 97–108, New York, NY, USA, 2019. Association for Computing Machinery.
- [79] I. NumFOCUS. Pandas. <https://pandas.pydata.org/>, 2022.
- [80] J. Östlund, T. Wrigstad, D. Clarke, and B. Åkerblom. Ownership, uniqueness, and immutability. In *International Conference on Software Technology: Methods and Tools*, 2008.
- [81] G. Ottoni, R. Rangan, A. Stoler, and D. I. August. Automatic thread extraction with decoupled software pipelining. In *38th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'05)*, pages 12–pp. IEEE, 2005.
- [82] A. Ousterhout, J. Fried, J. Behrens, A. Belay, and H. Balakrishnan. Shenango: Achieving high CPU efficiency for latency-sensitive datacenter workloads. In *NSDI*, pages 361–378, 2019.
- [83] A. Ousterhout, J. Fried, J. Behrens, A. Belay, and H. Balakrishnan. Shenango thread context switching. <https://github.com/shenango/shenango/blob/master/runtime/sched.c#L386>, 2023.
- [84] J. Ousterhout. Why Threads Are A Bad Idea (for most purposes). Technical report, USENIX Annual Technical Conference, 1996.
- [85] D. A. Padua and M. J. Wolfe. Advanced compiler optimizations for supercomputers. *Communications of the ACM*, 29(12):1184–1201, 1986.
- [86] V. S. Pai. Flash: An efficient and portable web server. In *1999 USENIX Annual Technical Conference (USENIX ATC 99)*, Monterey, CA, June 1999. USENIX Association.
- [87] G. Prekas, M. Kogias, and E. Bugnion. Zygos: Achieving low tail latency for microsecond-scale networked tasks. In *Proceedings of the 26th Symposium on Operating Systems Principles, SOSP '17*, page 325–341, New York, NY, USA, 2017. Association for Computing Machinery.
- [88] Y. Qiao, C. Wang, Z. Ruan, A. Belay, Q. Lu, Y. Zhang, M. Kim, and G. H. Xu. Hermit: Low-latency, high-throughput, and transparent remote memory via feedback-directed asynchrony. In *NSDI*, 2023.

- [89] P. Raju, R. Kadekodi, V. Chidambaram, and I. Abraham. Pebblesdb: Building key-value stores using fragmented log-structured merge trees. In *Proceedings of the 26th Symposium on Operating Systems Principles, SOSP '17*, page 497–514, New York, NY, USA, 2017. Association for Computing Machinery.
- [90] E. Raman, G. Ottoni, A. Raman, M. J. Bridges, and D. I. August. Parallel-stage decoupled software pipelining. In *Proceedings of the 6th annual IEEE/ACM international symposium on Code generation and optimization*, pages 114–123, 2008.
- [91] R. A. Rossi and N. K. Ahmed. The network data repository with interactive graph analytics and visualization. In *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, 2015.
- [92] Z. Ruan, M. Schwarzkopf, M. K. Aguilera, and A. Beley. AIFM: High-performance, application-integrated far memory. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 315–332, 2020.
- [93] S. M. Rumble, A. Kejriwal, and J. Ousterhout. Log-structured memory for dram-based storage. In *Proceedings of the 12th USENIX Conference on File and Storage Technologies, FAST'14*, pages 1–16, USA, 2014. USENIX Association.
- [94] S. Schneider, C. D. Antonopoulos, and D. S. Nikolopoulos. Scalable locality-conscious multi-threaded memory allocation. In *Proceedings of the 5th International Symposium on Memory Management, ISMM '06*, page 84–94, New York, NY, USA, 2006. Association for Computing Machinery.
- [95] Y. Shan, Y. Huang, Y. Chen, and Y. Zhang. LegoOS: A disseminated, distributed OS for hardware resource disaggregation. In *OSDI*, pages 69–87, 2018.
- [96] T. Sherwood, S. Sair, and B. Calder. Predictor-directed stream buffers. In *Proceedings 33rd Annual IEEE/ACM International Symposium on Microarchitecture. MICRO-33 2000*, pages 42–53, 2000.
- [97] J. Shun and G. E. Blelloch. Ligra: a lightweight graph processing framework for shared memory. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP '13*, page 135–146, New York, NY, USA, 2013. Association for Computing Machinery.
- [98] M. Sridharan, S. Chandra, J. Dolby, S. J. Fink, and E. Yahav. Alias analysis for object-oriented programs. *Aliasing in Object-Oriented Programming. Types, Analysis and Verification*, pages 196–232, 2013.
- [99] R. Stets, S. Dwarkadas, N. Hardavellas, G. Hunt, L. Kontothanassis, S. Parthasarathy, and M. Scott. Cashmere-2l: software coherent shared memory on a clustered remote-write network. In *Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles, SOSP '97*, page 170–183, New York, NY, USA, 1997. Association for Computing Machinery.
- [100] S. Stork, K. Naden, J. Sunshine, M. Mohr, A. Fonseca, P. Marques, and J. Aldrich. æminium: A permission-based concurrent-by-default programming language approach. *ACM Trans. Program. Lang. Syst.*, 36(1), mar 2014.
- [101] A. S. Tanenbaum and H. Bos. Modern operating systems. 2015.
- [102] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. Canton-Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, abs/2307.09288, 2023.
- [103] S.-Y. Tsai and Y. Zhang. LITE kernel RDMA support for datacenter applications. In *SOSP*, pages 306–324, 2017.
- [104] J. Tuck, L. Ceze, and J. Torrellas. Scalable cache miss handling for high memory-level parallelism. In *2006 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'06)*, pages 409–422, 2006.
- [105] R. von Behren, J. Condit, and E. Brewer. Why events are a bad idea (for High-Concurrency servers). In *9th Workshop on Hot Topics in Operating Systems (HotOS IX)*, Lihue, HI, May 2003. USENIX Association.
- [106] C. Wang, H. Ma, S. Liu, Y. Li, Z. Ruan, K. Nguyen, M. D. Bond, R. Netravali, M. Kim, and G. H. Xu. Semeru: A memory-disaggregated managed runtime. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 261–280. USENIX Association, Nov. 2020.

- [107] C. Wang, H. Ma, S. Liu, Y. Qiao, J. Eyolfson, C. Navasca, S. Lu, and G. H. Xu. MemLiner: Lining up tracing and application for a Far-Memory-Friendly runtime. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 35–53, 2022.
- [108] C. Wang, Y. Qiao, H. Ma, S. Liu, Y. Zhang, W. Chen, R. Netravali, M. Kim, and G. H. Xu. Canvas: Isolated and adaptive swapping for multi-applications on remote memory. In *NSDI*, 2023.
- [109] C. Wang, Y. Shan, P. Zuo, and H. Cui. Reinvent cloud software stacks for resource disaggregation. *Journal of Computer Science and Technology*, 38(5):949–969, 2023.
- [110] S. Westrick, M. Fluet, M. Rainey, and U. A. Acar. Automatic parallelism management. *Proc. ACM Program. Lang.*, 8(POPL), jan 2024.
- [111] J. Yang, Z. Qiu, Y. Zhang, Y. Yue, and K. V. Rashmi. Fifo can be better than lru: the power of lazy promotion and quick demotion. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems, HOTOS '23*, page 70–79, New York, NY, USA, 2023. Association for Computing Machinery.
- [112] W. Yoon, J. Ok, J. Oh, S. Moon, and Y. Kwon. Dilos: Do not trade compatibility for performance in memory disaggregation. In *Proceedings of the Eighteenth European Conference on Computer Systems, EuroSys '23*, page 266–282, New York, NY, USA, 2023. Association for Computing Machinery.
- [113] H. Zhu, Y. Chen, and X.-H. Sun. Timing local streams: improving timeliness in data prefetching. In *Proceedings of the 24th ACM International Conference on Supercomputing, ICS '10*, page 169–178, New York, NY, USA, 2010. Association for Computing Machinery.
- [114] X. Zhu, G. Feng, M. Serafini, X. Ma, J. Yu, L. Xie, A. Aboulnaga, and W. Chen. Livegraph: a transactional graph storage system with purely sequential adjacency list scans. *Proc. VLDB Endow.*, 13(7):1020–1034, mar 2020.